STUDENT RECORD MANAGEMENT SYSTEM

A C Language Mini Project

A PROJECT REPORT

Submitted by:

Kunal Chaudhary (26BCD10241)

Submitted to:

Name : Ms. Khushi Aggarwal

Designation: Assistant Professor

in partial fulfillment for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

Chandigarh University

October 2026

Submission Date: 26/Oct/2026

ABSTRACT

This project report presents the Student Record Management System, a menu-driven mini project developed in the C programming language. The program stores the details of students in an array of structures and lets the user manage them through a simple console menu.

Six core operations are implemented: adding a new student record, displaying all records in a table, searching a student by roll number, updating the marks of a student, deleting a record, and finding the topper of the class. For every student the program automatically calculates the total marks, the percentage and the grade.

The project uses basic C concepts such as structures, arrays, functions, pointers, loops, decision making and the switch statement. It is compiled with GCC and the real console output of every operation is included in this report.

TABLE OF CONTENTS

Contents	Page
Abstract	3
Table of Contents	4
Chapter 1 – Introduction	5
Chapter 2 – Objectives / Problem Statement	6
Chapter 3 – Modules, Tools and Technologies Used	7
Chapter 4 – Source Code with Output	9
Chapter 5 – Conclusion and Future Scope	17
References	18

CHAPTER 1. INTRODUCTION

1.1 About the Project

Every school and college has to keep records of its students, such as roll number, name and marks. When this work is done on paper it takes a lot of time, and it is easy to make mistakes while calculating totals, percentages and grades. A small computer program can do the same work quickly and correctly.

The Student Record Management System is a simple console application written in C. The user chooses an option from a menu, and the program performs the selected task on the student records stored in memory. It is an easy project, but it uses many of the important topics of the C language in one place.

1.2 Need of the Project

- **Saves time:** Manual calculation of total, percentage and grade is slow and error-prone; the program does it automatically.
- **Easy searching:** A record can be found by roll number in a single step instead of searching through pages.
- **Easy changes:** Marks can be corrected and wrong records can be removed without rewriting everything.
- **Learning value:** The project gives practice of structures, arrays, functions and pointers, which are core topics of C.

1.3 Scope

The program can store up to 50 students with three subjects each. Records are kept in memory while the program runs. The program is written in standard C99 and runs on any system that has a C compiler such as GCC.

CHAPTER 2. OBJECTIVES / PROBLEM STATEMENT

2.1 Problem Statement

To design and develop a menu-driven C program that can store, display, search, update and delete the records of students, automatically calculate the total marks, percentage and grade of each student, and identify the topper of the class.

2.2 Objectives

- Create a structure to hold roll number, name, marks, total, percentage and grade of a student.
- Add new students and reject a roll number that already exists.
- Display all student records in a neat table.
- Search a student by roll number and show the full result.
- Update the marks of a student and recalculate the result automatically.
- Delete a student record and shift the remaining records correctly.
- Find and display the topper of the class.
- Show proper messages for wrong choices or records that are not found.

2.3 Grading Rule Used

Percentage	Grade
90 and above	A
75 to less than 90	B
60 to less than 75	C
40 to less than 60	D
Below 40	F

CHAPTER 3. MODULES, TOOLS AND TECHNOLOGIES USED

3.1 Data Structure Used

All details of one student are kept together in a structure named Student. An array of 50 such structures stores the whole class, and a global variable count tells how many records are currently stored.

```
struct Student {
    int   roll;
    char  name[40];
    float marks[NUM_SUBJECTS];    /* NUM_SUBJECTS = 3 */
    float total;
    float percentage;
    char  grade;
};

struct Student students[MAX_STUDENTS];  /* MAX_STUDENTS = 50 */
int count = 0;
```

3.2 Modules of the Project

Module / Function	Purpose
addStudent()	Reads roll number, name and marks; rejects duplicate roll numbers; stores the record.
displayAll()	Prints all records in a table with percentage and grade.
searchStudent()	Finds a student by roll number and shows total, percentage and grade.
updateStudent()	Takes new marks for a student and recalculates the result.
deleteStudent()	Removes a record and shifts the later records one place up.
showTopper()	Finds the student with the highest percentage.
findIndex()	Helper: returns the position of a roll number in the array (or -1).
calculateResult()	Helper: calculates total, percentage and grade using a pointer to the structure.
findGrade()	Helper: returns the grade letter for a given percentage.
main()	Shows the menu in a do-while loop and calls the module chosen with switch-case.

3.3 Tools and Technologies Used

Item	Details
Programming language	C (C99 standard)
Compiler	GCC (GNU Compiler Collection)
Command used	gcc -std=c99 -Wall -Wextra -o student_record student_record.c
Header files	stdio.h (input / output), string.h (string handling)
C concepts used	Structures, arrays, functions, pointers, loops, if-else, switch-case, do-while
Code editor	Any text editor / IDE (VS Code, Code::Blocks, Dev-C++)
Version control	Git and GitHub for storing the project
Documentation	Microsoft Word

3.4 Working of the Program

When the program starts it prints a heading and then repeats the main menu until the user chooses Exit. The user types a number from 1 to 7; the switch statement calls the matching function. After the function finishes, the menu is shown again. If a number other than 1 to 7 is typed, the message "Invalid choice" is printed.

CHAPTER 4. SOURCE CODE WITH OUTPUT

4.1 Source Code (student_record.c)

Compile with: gcc -std=c99 -Wall -Wextra -o student_record student_record.c Run with: ./student_record

```
/*
 * ============================================================
 *  Project  : Student Record Management System
 *  Language : C (C99)
 *  Author   : Kunal Chaudhary (UID: 26BCA10241)
 *  Date     : 26 October 2026
 *  Compiler : GCC
 *
 *  A simple menu-driven program that stores student records
 *  in an array of structures. It supports adding, displaying,
 *  searching, updating and deleting records, and finding the
 *  topper of the class.
 * ============================================================
 */

#include <stdio.h>
#include <string.h>

#define MAX_STUDENTS 50
#define NUM_SUBJECTS 3

/* ---------- Structure that stores one student ---------- */
struct Student {
    int   roll;
    char  name[40];
    float marks[NUM_SUBJECTS];
    float total;
    float percentage;
    char  grade;
};

struct Student students[MAX_STUDENTS];
int count = 0;

/* ---------- Helper: decide grade from percentage ---------- */
char findGrade(float p) {
    if (p >= 90) return 'A';
    else if (p >= 75) return 'B';
    else if (p >= 60) return 'C';
    else if (p >= 40) return 'D';
    else return 'F';
}

/* ---------- Helper: find index of a roll number ---------- */
int findIndex(int roll) {
    for (int i = 0; i < count; i++) {
        if (students[i].roll == roll)
            return i;
    }
    return -1;
}

/* ---------- Helper: calculate total, percentage, grade ---------- */
void calculateResult(struct Student *s) {
    s->total = 0;
    for (int i = 0; i < NUM_SUBJECTS; i++)
        s->total += s->marks[i];
    s->percentage = s->total / NUM_SUBJECTS;
    s->grade = findGrade(s->percentage);
}
```

```
/* ---------- Module 1: Add a student ---------- */
void addStudent(void) {
    if (count >= MAX_STUDENTS) {
        printf("Record list is full!\n");
        return;
    }

    struct Student s;
    printf("Enter roll number : ");
    scanf("%d", &s.roll);

    if (findIndex(s.roll) != -1) {
        printf("Roll number already exists!\n");
        return;
    }

    printf("Enter name        : ");
    scanf(" %39[^\n]", s.name);

    for (int i = 0; i < NUM_SUBJECTS; i++) {
        printf("Enter marks of subject %d : ", i + 1);
        scanf("%f", &s.marks[i]);
    }

    calculateResult(&s);
    students[count++] = s;
    printf("Student record added successfully.\n");
}

/* ---------- Module 2: Display all students ---------- */
void displayAll(void) {
    if (count == 0) {
        printf("No records found.\n");
        return;
    }

    printf("\n%-6s %-18s %-6s %-6s %-6s %-10s %-5s\n",
           "Roll", "Name", "M1", "M2", "M3", "Percent", "Grade");
    printf("-----------------------------------------------------------------\n");
    for (int i = 0; i < count; i++) {
        printf("%-6d %-18s %-6.0f %-6.0f %-6.0f %-10.2f %-5c\n",
               students[i].roll, students[i].name,
               students[i].marks[0], students[i].marks[1],
               students[i].marks[2], students[i].percentage,
               students[i].grade);
    }
}

/* ---------- Module 3: Search a student ---------- */
void searchStudent(void) {
    int roll;
    printf("Enter roll number to search : ");
    scanf("%d", &roll);

    int i = findIndex(roll);
    if (i == -1) {
        printf("Student with roll number %d not found.\n", roll);
        return;
    }

    printf("\nRecord found:\n");
    printf("Roll       : %d\n", students[i].roll);
    printf("Name       : %s\n", students[i].name);
    printf("Total      : %.0f / %d\n", students[i].total, NUM_SUBJECTS * 100);
    printf("Percentage : %.2f%%\n", students[i].percentage);
    printf("Grade      : %c\n", students[i].grade);
}

/* ---------- Module 4: Update marks ---------- */
void updateStudent(void) {
```

```
    int roll;
    printf("Enter roll number to update : ");
    scanf("%d", &roll);

    int i = findIndex(roll);
    if (i == -1) {
        printf("Student with roll number %d not found.\n", roll);
        return;
    }

    for (int j = 0; j < NUM_SUBJECTS; j++) {
        printf("Enter new marks of subject %d : ", j + 1);
        scanf("%f", &students[i].marks[j]);
    }
    calculateResult(&students[i]);
    printf("Record updated successfully.\n");
}

/* ---------- Module 5: Delete a student ---------- */
void deleteStudent(void) {
    int roll;
    printf("Enter roll number to delete : ");
    scanf("%d", &roll);

    int i = findIndex(roll);
    if (i == -1) {
        printf("Student with roll number %d not found.\n", roll);
        return;
    }

    for (int j = i; j < count - 1; j++)
        students[j] = students[j + 1];
    count--;
    printf("Record deleted successfully.\n");
}

/* ---------- Module 6: Show the topper ---------- */
void showTopper(void) {
    if (count == 0) {
        printf("No records found.\n");
        return;
    }

    int top = 0;
    for (int i = 1; i < count; i++) {
        if (students[i].percentage > students[top].percentage)
            top = i;
    }
    printf("\nTopper of the class: %s (Roll %d) with %.2f%%\n",
           students[top].name, students[top].roll, students[top].percentage);
}

/* ---------- Main menu ---------- */
int main(void) {
    int choice;

    printf("=======================================\n");
    printf("   STUDENT RECORD MANAGEMENT SYSTEM\n");
    printf("=======================================\n");

    do {
        printf("\n1. Add Student\n");
        printf("2. Display All Students\n");
        printf("3. Search Student\n");
        printf("4. Update Student Marks\n");
        printf("5. Delete Student\n");
        printf("6. Show Topper\n");
        printf("7. Exit\n");
        printf("Enter your choice : ");

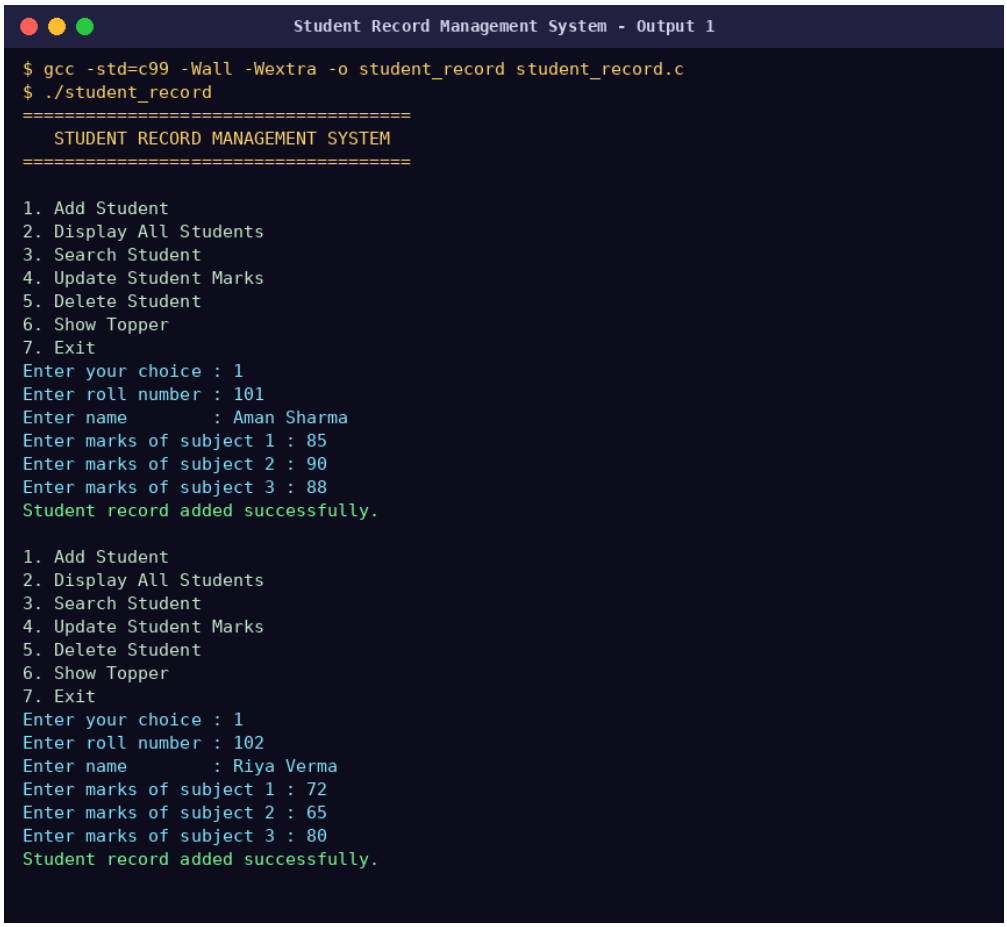
        if (scanf("%d", &choice) != 1)
            break;
```

```
        switch (choice) {
            case 1: addStudent();    break;
            case 2: displayAll();    break;
            case 3: searchStudent(); break;
            case 4: updateStudent(); break;
            case 5: deleteStudent(); break;
            case 6: showTopper();    break;
            case 7: printf("\nThank you for using the system. Goodbye!\n"); break;
            default: printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 7);

    return 0;
}
```

4.2 Output Screenshots

The screenshots below show the output of the program compiled with GCC and run in the terminal. The input typed by the user (such as the choice number, roll number and marks) is visible after each prompt.



Output 1: Program heading, main menu, and adding the first two students (roll 101 and 102).

```
Student Record Management System - Output 2

1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 1
Enter roll number : 103
Enter name        : Kunal Singh
Enter marks of subject 1 : 95
Enter marks of subject 2 : 92
Enter marks of subject 3 : 98
Student record added successfully.

1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 1
Enter roll number : 104
Enter name        : Neha Gupta
Enter marks of subject 1 : 55
Enter marks of subject 2 : 48
Enter marks of subject 3 : 60
Student record added successfully.
```

Output 2: Adding the third and fourth students (roll 103 and 104).

```
Student Record Management System - Output 3

1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 2

Roll   Name              M1     M2     M3     Percent    Grade
-----------------------------------------------------------------
101    Aman Sharma       85     90     88     87.67      B
102    Riya Verma        72     65     80     72.33      C
103    Kunal Singh       95     92     98     95.00      A
104    Neha Gupta        55     48     60     54.33      D

1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 3
Enter roll number to search : 102

Record found:
Roll       : 102
Name       : Riya Verma
Total      : 217 / 300
Percentage : 72.33%
Grade      : C
```

Output 3: Displaying all students in a table with percentage and grade.

Student Record Management System - Output 4

```
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 4
Enter roll number to update : 104
Enter new marks of subject 1 : 70
Enter new marks of subject 2 : 65
Enter new marks of subject 3 : 72
Record updated successfully.

1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 2

Roll   Name              M1     M2     M3     Percent    Grade
-------------------------------------------------------------
101    Aman Sharma       85     90     88     87.67      B
102    Riya Verma        72     65     80     72.33      C
103    Kunal Singh       95     92     98     95.00      A
104    Neha Gupta        70     65     72     69.00      C
```

Output 4: Searching roll number 102 and showing the full record.

Student Record Management System - Output 5

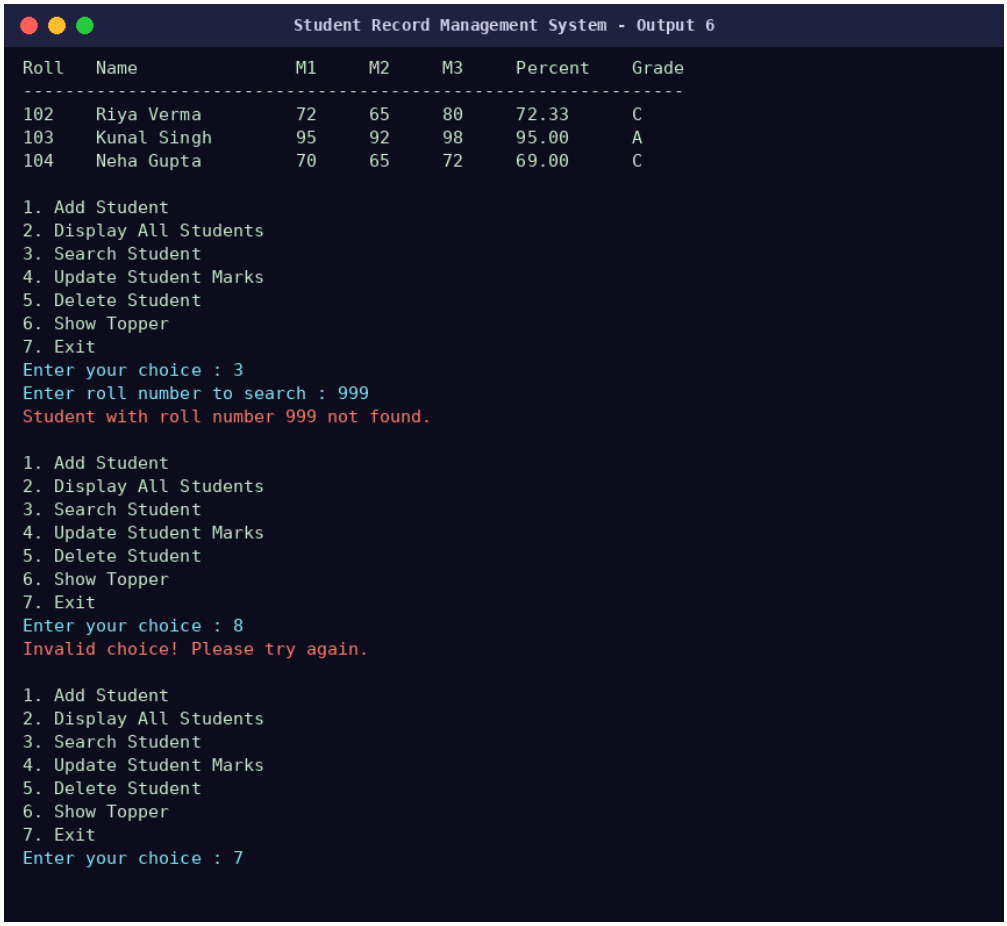
```
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 6

Topper of the class: Kunal Singh (Roll 103) with 95.00%

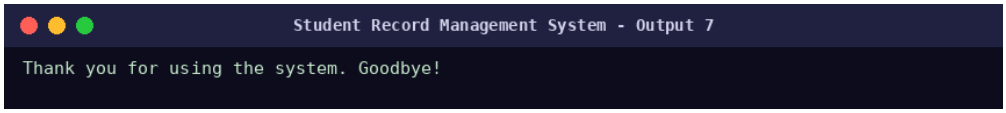
1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 5
Enter roll number to delete : 101
Record deleted successfully.

1. Add Student
2. Display All Students
3. Search Student
4. Update Student Marks
5. Delete Student
6. Show Topper
7. Exit
Enter your choice : 2
```

Output 5: Updating the marks of roll number 104 and displaying the table again.



Output 6: Finding the topper and deleting the record of roll number 101.



Output 7: Table after deletion, searching a roll number that does not exist, an invalid menu choice, and Exit.

4.3 Test Cases

Test	Input	Expected	Result
Add student	Roll 101, marks 85, 90, 88	Added; percentage 87.67, grade B	Pass
Add student	Roll 103, marks 95, 92, 98	Added; percentage 95.00, grade A	Pass
Search	Roll 102	Record shown, grade C	Pass
Update	Roll 104, marks 70, 65, 72	Percentage becomes 69.00, grade C	Pass
Topper	4 records stored	Kunal Singh (Roll 103), 95.00%	Pass
Delete	Roll 101	Record removed, 3 records remain	Pass
Search missing	Roll 999	"not found" message	Pass
Invalid choice	8	"Invalid choice" message	Pass

CHAPTER 5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Student Record Management System was successfully developed in C and tested with GCC. All six operations – add, display, search, update, delete and topper – work correctly and give the expected output. Total marks, percentage and grade are calculated automatically, which removes calculation mistakes. Wrong inputs such as a repeated roll number, a missing record or an invalid menu choice are handled with clear messages.

Through this project the important topics of C such as structures, arrays, functions, pointers, loops and the switch statement were practised in a real, useful program. The code is divided into small functions, so it is easy to read and easy to extend.

5.2 Future Scope

- Save records in a file so that data is not lost when the program closes.
- Add more subjects and let the user choose the number of subjects.
- Sort the students by marks or by name and print a rank list.
- Add a login system for teachers.
- Build a graphical interface or a database-backed version.

5.3 Github

https://github.com/karanchaudhary292007-dev/STUDENT_RECORD_MANAGEMENT_SYSTEM.git